

```
except ValueError:
    return 100
finally:
    print("cleanup")
```

print(fun(-5))

- A)100
- cleanup
- B)cleanup
- 100
- C)negative
- cleanup
- D)cleanup
- 5

Q47. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        global x
        x = 30
```

```
    inner()
    print(x)
```

```
outer()
print(x)
```

- A)20
- 10
- B)30
- 30
- C)30
- 20
- D)20
- 30

Q48. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)Wrong Value
- B)ValueError
- C)e
- D)Nothing

Q49. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

- A)There is no difference
- B)Both import only sqrt
- C)First provides the module namespace; second imports sqrt directly
- D)First imports only sqrt

Q50. What will be printed?

```
try:
    x = int("10")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)B C
- B)A
- C)A C
- D)B



```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)A
- B)B C
- C)B
- D)A C

Q36. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

- A)Only Hello
- B)10
Hello
- C)x
display
- D)Error

Q37. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
```

- A)Error
- B)[10]
[10, 10]
- C)[10]
[10]

```
D)[10, 10]
[10]
```

Q38. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        return x + 5
```

```
    return inner
```

```
f = outer()
print(f())
```

- A)15
- B)NameError
- C)10
- D)None

Q39. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

```
print(fun())
```

- A)A
- B
- D
- 10
- B)A
- C
- D
- 20
- C)A
- B
- 10
- D



```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A)Something went wrong
- B)TypeError
- C)MyError
- D)Exception

Q21. What will normally happen?

```
import math
import math
```

- A)SyntaxError
- B)ImportError
- C)The module is normally loaded and executed twice
- D)The module is normally loaded once and reused

Q22. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

- A)SyntaxError
- B)20
- C)30
- D)TypeError

Q23. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)NameError
- B)20
- C)10
- D)None

Q24. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)6
- B)None
- C)3
- D)5

**Q25. if test.py has
print("A")**

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A followed by B
- B)Nothing
- C)A
- D)B

Q26. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)UnboundLocalError
- B)50
- C)10
- D)NameError

Q27. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

- A)NameError
- B)TypeError
- C)0
- D)None

Q28. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
```



```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Error
- B)None
- C)Hello
- D)Hello None

Q7. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

- A)20
- B)NameError
- C)UnboundLocalError
- D)10

Q8. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)A
- B)B
- C)A B
- D)Nothing

Q9. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

- A)SyntaxError
- B)20
- C)30
- D)TypeError

Q10. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

- A)TypeError
- B)6
- C)RecursionError
- D)3

Q11. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

- A)B
- B)Nothing
- C)A
- D)A B

Q12. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

- A)1
- 2
- 3
- 4
- B)1
- 1
- 1



```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

A)[2, 4, 6, 8, 10]

B)[4, 8]

C)[1, 3, 5]

D)[2, 4]

Q47. Why is

try:

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)except blocks execute randomly

B)ZeroDivisionError occurs before Exception

C)Exception is a superclass of ZeroDivisionError

D)ZeroDivisionError is not an exception

Q48. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)10

B)NameError

C)20

D)30

Q49. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)random cannot be imported

B)Python will always reject it

C)The local file can shadow the intended standard-library module

D)Python automatically renames the local file

Q50. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)NameError

B)10

C)20

D)SyntaxError



except ValueError:

print("A")

else:

print("B")

finally:

print("C")

A)A

B)B C

C)B

D)A C

Q35. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

A)TypeError

B)SyntaxError

C)30

D)20

Q36. What happens when this code runs?

def first():

second()

def second():

third()

def third():

raise RuntimeError("Failure")

try:

first()

except RuntimeError:

print("Handled")

A)Handled

B)RuntimeError cannot propagate through functions

C)NameError

D)TypeError

Q37. What is the output?

def fun():

try:

return 10

finally:

return 20

print(fun())

A)RuntimeError

B)10

C)None

D)20

Q38. What is the output?

def fun():

try:

print("A")

raise ValueError

except ValueError:

print("B")

return 10

else:

print("C")

return 20

finally:

print("D")

print(fun())

A)A

B

10

D

B)A

B

C

D

10

C)A

B

D

10

D)A

C

D

20

Q39. What will be printed?

def fun(x=[]):

x.append(10)

return x

a = fun()

b = fun()



print(a())

A)1

2

3

4

B)0

1

0

2

C)1

2

1

3

D)1

1

1

1

Q21. What is the output?

try:

raise ValueError

except ValueError:

print("A")

raise

A)A, and execution ends normally

B)B

C)A, followed by propagation of the same exception

D)No Output

Q22. What happens when this function is called?

def fun(a=10, b):

return a + b

A)It executes correctly

B)NameError

C)SyntaxError

D)TypeError

Q23. What does the following function return?

def fun():

print("Hello")

x = fun()

print(x)

A>Error

B)Hello None

C)Hello

D)None

Q24. Suppose module_a.py contains:

print("Module loaded")

value = 10

And another file contains:

import module_a

import module_a

print(module_a.value)

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

Module loaded

10

B)ImportError

C)Module loaded

10

D)10

10

Q25. What is the purpose of this statement?

if __name__ == "__main__":

A)It prevents all imports

B)It identifies code intended to run when the file is executed directly

C)It creates the main function

D)It prevents syntax errors

Q26. What is the output?

def fun(x):

return x * 2

a = fun

print(a(5))

A)func(5)

B>Error

C)10

D)5



fun()

A)50

B)10

C)NameError

D)UnboundLocalError

Q8. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)15

B)5

C>Error

D)10

Q9. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)B

C)B C

D)A

Q10. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)NameError

B)6

C)36

D)6.0

Q11. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

A)5 15

B)15 5

C)15 15

D)5 5

Q12. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

A)ValueError

B)e

C)Nothing

D)Wrong Value

Q13. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)9

B)24

C)0

D)10

Q14. What will be printed?

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is
too low")
```

```
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```



A)Nothing

B)A

C)A B

D)B

Q47. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

A)Both import only sqrt

B)First provides the module namespace; second imports sqrt directly

C)First imports only sqrt

D)There is no difference

Q48. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)A

C)B

D)B C

Q49. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)25

B)sqrt(25)

C)5.0

D)5

Q50. What will be printed?

```
def make_counter():
```

```
    count = 0
```

```
    def counter():
```

```
        nonlocal count
```

```
        count += 1
```

return count

return counter

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)0

1

0

2

B)1

2

1

3

C)1

2

3

4

D)1

1

1

1



C)TypeError
D)NameError

Q33. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)B
B)A and B
C)A
D)Nothing

Q34. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)30
B)SyntaxError
C)TypeError
D)20

Q35. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)5
B)15
C)Error
D)10

Q36. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)  
A)5 15
```

B)15 5
C)15 15
D)5 5

Q37. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

A)TypeError
B)25
C)15
D)SyntaxError

Q38. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)SyntaxError
B)30
C)20
D)TypeError

Q39. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)10
10
B)Module loaded
10
C)ImportError
D)Module loaded
Module loaded
10



D)Nothing

Q21. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)InvalidAgeError

15

B)Exception

Age is too low

C)InvalidAgeError

Age is too low

D)Nothing, because custom exceptions cannot be caught

Q22. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

A)30

20

B)20

30

C)20

10

D)30

30

Q23. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)random cannot be imported

B)Python will always reject it

C)The local file can shadow the intended standard-library module

D)Python automatically renames the local file

Q24. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```

A)10 only

B)Finally only

C)Finally followed by 10

D)10 followed by Finally

Q25. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)10

B)20

C)None

D)NameError

Q26. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():
```



```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)None
- B)Hello
- C>Error
- D>Hello None

Q8. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.modules
- B)sys.imports
- C)sys.path
- D)__module__

Q9. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

```
print(fun(-5))
```

- A)negative
cleanup
- B)100
cleanup
- C)cleanup
-5
- D)cleanup
100

Q10. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")
```

```
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

```
print(fun())
```

- A)A
- B
- D
- 10
- B)A
- B
- C
- D
- 10
- C)A
- B
- 10
- D
- D)A
- C
- D
- 20

Q11. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

```
print(fun())
```

- A)None
- B)20
- C)RuntimeError
- D)10

Q12. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6
- B)6.0
- C)NameError
- D)36



C)0

1

0

2

D)1

2

1

3

Q46. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

A)B

B)A B

C)A

D)Nothing

Q47. What is the major difference between the following?

import math

and

from math import sqrt

A)First imports only sqrt

B)There is no difference

C)First provides the module namespace; second imports sqrt directly

D)Both import only sqrt

Q48. What does the following function return?

```
def fun():  
    print("Hello")
```

x = fun()

print(x)

A>Error

B>Hello

C>Hello None

D)None

Q49. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

a = fun()

b = fun()

print(a)

print(b)

A>Error

B)[10]

[10, 10]

C)[10, 10]

[10]

D)[10]

[10]

Q50. What will happen?

```
def fun(a, b=10):  
    return a + b
```

print(fun(b=5, a=20))

A)SyntaxError

B)TypeError

C)15

D)25



```

map(
    lambda x: x * 2,
    filter(lambda x: x % 2 == 0,
numbers)
)
)

```

print(result)

A)[2, 4, 6, 8, 10]

B)[2, 4]

C)[4, 8]

D)[1, 3, 5]

Q33. What happens here?

try:

```

    raise ValueError("Wrong value")

```

except ValueError as e:

```

    print(e)

```

A)Wrong Value

B)ValueError

C)Nothing

D)e

Q34. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)Python automatically renames the local file

C)The local file can shadow the intended standard-library module

D)Python will always reject it

Q35. What happens here?

```

class MyError(Exception):
    pass

```

def test():

```

    raise MyError("Something went wrong")

```

try:

```

    test()

```

except MyError as e:

```

    print(type(e).__name__)

```

A)TypeError

B)Something went wrong

C)MyError

D)Exception

Q36. Why is

try:

```

    print(10 / 0)

```

except Exception:

```

    print("A")

```

except ZeroDivisionError:

```

    print("B")

```

codes's second handler never reached?

A)ZeroDivisionError occurs before Exception

B)ZeroDivisionError is not an exception

C)Exception is a superclass of ZeroDivisionError

D)except blocks execute randomly

Q37. What happens when this function is called?

```

def fun(a=10, b):

```

```

    return a + b

```

A)It executes correctly

B)TypeError

C)SyntaxError

D)NameError

Q38. What is the output?

```

def fun(a=10, b=20):

```

```

    return a - b

```

```

print(fun(30))

```

A)30

B)-10

C)20

D)10

Q39. Which object keeps track of modules that have already been imported during the current Python process?

A)__module__

B)sys.imports

C)sys.path

D)sys.modules

Q40. What is the output?

```

def outer():

```



```
print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

- A)Exception
- Age is too low
- B)InvalidAgeError
- 15
- C)InvalidAgeError
- Age is too low
- D)Nothing, because custom exceptions cannot be caught

Q19. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

- A)10
- B)0
- C)24
- D)9

Q20. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

- A)15 5
- B)5 5
- C)5 15
- D)15 15

Q21. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents syntax errors

B)It identifies code intended to run when the file is executed directly

C)It creates the main function

D)It prevents all imports

Q22. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

- A)15
- B)NameError
- C)10
- D)None

Q23. What will normally happen?

```
import math
```

```
import math
```

- A)The module is normally loaded and executed twice
- B)The module is normally loaded once and reused
- C)SyntaxError
- D)ImportError

Q24. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6
- B)NameError
- C)36
- D)6.0

Q25. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)5



else:

print("B")

finally:

print("C")

A)A

B)B

C)B C

D)A C

Q7. Suppose sample.py contains:

x = 10

def display():

print("Hello")

What will this print?

import sample

print(sample.x)

sample.display()

A)Error

B)Only Hello

C)10

Hello

D)x

display

Q8. What is the output?

try:

raise ValueError

except ValueError:

print("A")

raise

A)A, followed by propagation of the same exception

B)B

C)No Output

D)A, and execution ends normally

Q9. What will be printed?

try:

print(10 / 0)

except ZeroDivisionError:

print("A")

except Exception:

print("B")

A)A B

B)Nothing

C)B

D)A

Q10. What will be the output?

x = 10

def fun():

print(x)

x = 20

fun()

A)UnboundLocalError

B)10

C)20

D)NameError

Q11. Suppose module_a.py contains:

print("Module loaded")

value = 10

And another file contains:

import module_a

import module_a

print(module_a.value)

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError

B)10

10

C)Module loaded

10

D)Module loaded

Module loaded

10

Q12. What is the output?

f = lambda x: x + 5

print(f(10))

A)5

B>Error

C)10



C)cleanup

100

D)cleanup

-5

Q48. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

1

3

B)1

2

3

4

C)0

1

0

2

D)1

1

1

1

Q49. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents all imports

B)It creates the main function

C)It identifies code intended to run when the file is executed directly

D)It prevents syntax errors

Q50. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)InvalidAgeError

15

B)Exception

Age is too low

C)Nothing, because custom exceptions cannot be caught

D)InvalidAgeError

Age is too low



return inner

```
f = outer()
print(f())
```

A)NameError
B)15
C)None
D)10

Q35. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

A)10
Hello
B)x
display
C)Error
D)Only Hello

Q36. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

A)SyntaxError
B)20
C)TypeError
D)30

Q37. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)15
B)Error
C)10
D)5

Q38. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)TypeError
B)RuntimeError cannot propagate through functions
C)Handled
D)NameError

Q39. What will happen?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal y
        y = 20
```

```
    inner()
```

```
outer()
```

A)20
B)NameError
C)10
D)SyntaxError

Q40. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

A)15 15
B)5 15
C)15 5



Q20. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

- A)None
- B)NameError
- C)0
- D)TypeError

Q21. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)random cannot be imported
- B)The local file can shadow the intended standard-library module
- C)Python automatically renames the local file
- D)Python will always reject it

Q22. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6
- B)36
- C)NameError
- D)6.0

Q23. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

```
print(fun())
```

- A)10
- B)20
- C)None
- D)RuntimeError

Q24. Suppose module_a.py contains:

```
print("Module loaded")
```

value = 10

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
- 10
- B)Module loaded
- Module loaded
- 10
- C)ImportError
- D)10

10

Q25. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[2, 4, 6, 8, 10]
- B)[4, 8]
- C)[1, 3, 5]
- D)[2, 4]

Q26. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

- A)10
- B)UnboundLocalError



wrong")

```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A)TypeError
- B)MyError
- C)Something went wrong
- D)Exception

Q7. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

print(fun(30))

- A)-10
- B)20
- C)10
- D)30

Q8. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)No Output
- B)A, followed by propagation of the same exception
- C)A, and execution ends normally
- D)B

Q9. What will happen?

```
def fun(a, b=10):
    return a + b
```

print(fun(b=5, a=20))

- A)SyntaxError
- B)TypeError
- C)25
- D)15

Q10. What is the output?

```
def fun(x):
    return x * 2
```

a = fun

print(a(5))

- A)10
- B)func(5)
- C>Error
- D)5

Q11. What will normally happen?

import math

import math

- A)ImportError
- B)The module is normally loaded and executed twice
- C)The module is normally loaded once and reused
- D)SyntaxError

Q12. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

print(fun())

- A)A
- B
- C
- D
- 10
- B)A
- B
- D
- 10
- C)A
- C
- D
- 20
- D)A
- B
- 10



C) __module__

D) sys.modules

Q46. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A) NameError

B) 10

C) 30

D) 20

Q47. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A) NameError

B) 10

C) SyntaxError

D) 20

Q48. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

A) 5 5

B) 5 15

C) 15 5

D) 15 15

Q49. What happens?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, a=20))
```

A) TypeError

B) SyntaxError

C) 20

D) 30

Q50. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A) ZeroDivisionError occurs before Exception

B) except blocks execute randomly

C) Exception is a superclass of ZeroDivisionError

D) ZeroDivisionError is not an exception



C)A
B
C
D
10
D)A
B
D
10

Q33. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)0
B)24
C)9
D)10

Q34. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

A)UnboundLocalError
B)15
C)10
D)5

Q35. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It identifies code intended to run when the file is executed directly
B)It prevents all imports
C)It creates the main function
D)It prevents syntax errors

Q36. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

A)[4, 8]
B)[1, 3, 5]
C)[2, 4]
D)[2, 4, 6, 8, 10]

Q37. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)A and B
B)B
C)Nothing
D)A

Q38. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6.0
B)36
C)6
D)NameError



print(fun(3))

- A)6
- B)3
- C)5
- D)None

Q19. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

- A)SyntaxError
- B)30
- C)TypeError
- D)20

Q20. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

- A)A, and execution ends normally
- B)A, followed by propagation of the same exception
- C)No Output
- D)B

Q21. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

- A)NameError
- B)20
- C)10
- D)None

**Q22. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)B

B)Nothing

C)A followed by B

D)A

Q23. What will normally happen?

import math

import math

- A)The module is normally loaded and executed twice
- B)SyntaxError
- C)The module is normally loaded once and reused
- D)ImportError

Q24. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)B
- B)A
- C)B C
- D)A C

Q25. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)RuntimeError
- B)20
- C)None
- D)10

Q26. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```



C)B

D)A B

Q7. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

A)RecursionError

B)6

C)3

D)TypeError

Q8. What happens here?

```
def fun(a):  
    return a
```

print(fun())

A)NameError

B)None

C)0

D)TypeError

Q9. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q10. What will be printed?

```
def make_counter():  
    count = 0
```

```
def counter():  
    nonlocal count  
    count += 1  
    return count
```

return counter

a = make_counter()

b = make_counter()

print(a())

print(a())

print(b())

print(a())

A)1

2

1

3

B)1

2

3

4

C)1

1

1

1

D)0

1

0

2

Q11. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

A)A B

B)SyntaxError

C)A



A)20

10

B)30

30

C)20

30

D)30

20

Q46. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

A)10

B)func(5)

C>Error

D)5

**Q47. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

A)A

B)A followed by B

C)B

D)Nothing

Q48. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

A)10

B)5

C)15

D)UnboundLocalError

Q49. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)TypeError

B)MyError

C)Exception

D)Something went wrong

Q50. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

A)A, and execution ends normally

B)No Output

C)A, followed by propagation of the same
exception

D)B



```
print(math.sqrt(25))
```

A)sqrt(25)

B)5.0

C)5

D)25

Q34. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B

C)A C

D)B C

Q35. What will be printed?

```
def fun(x=[]):
```

```
    x.append(10)
```

```
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```

```
print(b)
```

A)Error

B)[10]

[10]

C)[10]

[10, 10]

D)[10, 10]

[10]

Q36. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)TypeError

B)NameError

C)It executes correctly

D)SyntaxError

Q37. What will be printed?

```
class InvalidAgeError(Exception):
```

```
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is too low")
```

```
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```

A)Exception

Age is too low

B)InvalidAgeError

15

C)Nothing, because custom exceptions cannot be caught

D)InvalidAgeError

Age is too low

Q38. What will be printed?

```
try:
```

```
    print(10 / 0)
```

```
except ZeroDivisionError:
```

```
    print("A")
```

```
except Exception:
```

```
    print("B")
```

A)A

B)Nothing

C)B

D)A B

Q39. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)6

B)None



print("B")

fun()

- A)A
- B)Nothing
- C)B
- D)A B

Q20. What will be the output?

x = 10

def fun():

print(x)

x = 20

fun()

- A)20
- B)NameError
- C)10
- D)UnboundLocalError

Q21. What is the output?

def fun():

try:

print("A")

raise ValueError

except ValueError:

print("B")

return 10

else:

print("C")

return 20

finally:

print("D")

print(fun())

- A)A
- C
- D
- 20
- B)A
- B
- 10
- D
- C)A
- B
- D

10

D)A

B

C

D

10

Q22. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

- A)30
- B)SyntaxError
- C)TypeError
- D)20

Q23. What is the purpose of this statement?

if __name__ == "__main__":

- A)It identifies code intended to run when the file is executed directly
- B)It prevents syntax errors
- C)It creates the main function
- D)It prevents all imports

Q24. What will be the output?

def fun(x):

try:

if x < 0:

raise ValueError("negative")

return x

except ValueError:

return 100

finally:

print("cleanup")

print(fun(-5))

- A)cleanup
- 100
- B)cleanup
- 5
- C)negative
- cleanup
- D)100
- cleanup

Q25. What will happen?



B)Module loaded

Module loaded

10

C)10

10

D)Module loaded

10

Q7. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

```
print(result)
```

A)[1, 3, 5]

B)[4, 8]

C)[2, 4, 6, 8, 10]

D)[2, 4]

Q8. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)SyntaxError

B)NameError

C)10

D)20

Q9. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

A)e

B)Nothing

C)ValueError

D)Wrong Value

Q10. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)15

B)NameError

C)None

D)10

Q11. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)Finally followed by 10

B)Finally only

C)10 followed by Finally

D)10 only

Q12. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)ZeroDivisionError is not an exception

B)except blocks execute randomly

C)Exception is a superclass of ZeroDivisionError

D)ZeroDivisionError occurs before Exception

Q13. What happens?



print(f(10))

- A)10
- B)5
- C)15
- D)Error

Q46. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

- A)Error
- B)Only Hello
- C)x
display
- D)10
Hello

Q47. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

- A)No Output
- B)A, and execution ends normally
- C)A, followed by propagation of the same exception
- D)B

Q48. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)First imports only sqrt
- B)There is no difference

C)First provides the module namespace;
second imports sqrt directly

D)Both import only sqrt

Q49. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)A
- B)B C
- C)A C
- D)B

Q50. What will normally happen?

```
import math  
import math
```

- A)ImportError
- B)The module is normally loaded once and reused
- C)The module is normally loaded and executed twice
- D)SyntaxError



x = 10

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

outer()

- A)NameError
- B)10
- C)30
- D)20

Q34. What happens here?

```
def fun(a):  
    return a
```

print(fun())

- A)None
- B)NameError
- C)TypeError
- D)0

Q35. What will be the output?

```
import math
```

print(math.sqrt(25))

- A)25
- B)sqrt(25)
- C)5
- D)5.0

Q36. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)10
- B)UnboundLocalError
- C)20
- D)NameError

Q37. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

outer()

- A)NameError
- B)10
- C)20
- D)SyntaxError

Q38. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

- A)ZeroDivisionError is not an exception
- B)ZeroDivisionError occurs before Exception
- C)Exception is a superclass of ZeroDivisionError
- D)except blocks execute randomly

Q39. What will be printed?

```
def fun(x=[]):
```

```
    x.append(10)
```

```
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```

```
print(b)
```

- A>Error
- B)[10]
- [10, 10]
- C)[10, 10]
- [10]



D)10

Q20. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

A)20

B)RuntimeError

C)None

D)10

Q21. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

A)Nothing

B)Wrong Value

C)ValueError

D)e

Q22. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

x = 5

y = fun(x)

print(x, y)

A)15 15

B)15 5

C)5 15

D)5 5

Q23. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

A)SyntaxError

B)TypeError

C)30

D)20

Q24. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)InvalidAgeError

15

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

Age is too low

D)Exception

Age is too low

Q25. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

A)A

B

C

D

10



- A)None
- B)NameError
- C)10
- D)15

Q7. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)15
- B)SyntaxError
- C)TypeError
- D)25

Q8. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
Module loaded
10
- B)Module loaded
10
- C)ImportError
- D)10
10

Q9. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

- A)TypeError
- B)20
- C)30
- D)SyntaxError

Q10. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)None
- B)20
- C)10
- D)NameError

Q11. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)10
- B)-10
- C)20
- D)30

Q12. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)Nothing
- B)A and B
- C)A
- D)B

Q13. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```



print(fun(3))

A)None

B)3

C)5

D)6

Q46. What will happen?

def fun(n):

if n == 0:

return 1

return n * fun(n)

print(fun(3))

A)RecursionError

B)3

C)TypeError

D)6

Q47. What is the output?

def fun(x):

x += 10

return x

x = 5

y = fun(x)

print(x, y)

A)15 5

B)15 15

C)5 5

D)5 15

Q48. if test.py has

print("A")

if __name__ == "__main__":

print("B")

If test.py is executed directly, what will be printed?

A)A

B)Nothing

C)B

D)A followed by B

Q49. What will be the output?

def fun(x):

try:

if x < 0:

raise ValueError("negative")

return x

except ValueError:

return 100

finally:

print("cleanup")

print(fun(-5))

A)cleanup

100

B)negative

cleanup

C)100

cleanup

D)cleanup

-5

Q50. What is the output?

def fun(x):

return x * 2

a = fun

print(a(5))

A)10

B)func(5)

C>Error

D)5



```
from math import sqrt
```

A)First provides the module namespace;
second imports sqrt directly

B)There is no difference

C)First imports only sqrt

D)Both import only sqrt

Q31. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)10

B)None

C)NameError

D)20

Q32. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

A)NameError

B)UnboundLocalError

C)10

D)20

Q33. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B C

B)A C

C)A

D)B

Q34. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)Exception is a superclass of
ZeroDivisionError

B)except blocks execute randomly

C)ZeroDivisionError occurs before
Exception

D)ZeroDivisionError is not an exception

Q35. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)A

B)B

C)Nothing

D)A B

**Q36. What happens when this code
executes?**

```
def fun():
```

```
    print(variable)
```

```
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)10

B)50

C)NameError

D)UnboundLocalError

Q37. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```



return counter

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)0

1

0

2

B)1

2

1

3

C)1

1

1

1

D)1

2

3

4

Q19. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.imports

B)__module__

C)sys.path

D)sys.modules

Q20. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

A)30

B)-10

C)20

D)10

Q21. What does the following function return?

```
def fun():
```

```
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)Hello

B>Error

C)Hello None

D)None

Q22. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

A)20

B)SyntaxError

C)30

D)TypeError

Q23. What will be printed?

```
class InvalidAgeError(Exception):
    pass
```

```
def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

A)InvalidAgeError

15

B)InvalidAgeError

Age is too low

C)Nothing, because custom exceptions cannot be caught

D)Exception

Age is too low

Q24. What is the purpose of this statement?



```
finally:  
    print("D")
```

```
print(fun())
```

A)A

B

10

D

B)A

B

D

10

C)A

B

C

D

10

D)A

C

D

20

Q7. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)SyntaxError

C)TypeError

D)20

Q8. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)10

B)0

C)9

D)24

Q9. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)MyError

B)Something went wrong

C)Exception

D)TypeError

Q10. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

10

B)Module loaded

Module loaded

10

C)ImportError

D)10

10

Q11. What will be printed?

```
x = 10
```

```
def outer():  
    x = 20
```



except ValueError:

print("A")

else:

print("B")

finally:

print("C")

A)A

B)B C

C)B

D)A C

Q47. What will be printed?

class InvalidAgeError(Exception):

pass

def check_age(age):

if age < 18:

raise InvalidAgeError("Age is too low")

return "Allowed"

try:

print(check_age(15))

except InvalidAgeError as e:

print(type(e).__name__)

print(e)

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q48. What is the output?

def fun():

try:

return 10

finally:

return 20

print(fun())

A)RuntimeError

B)10

C)20

D)None

Q49. Suppose test.py contains:

print("A")

if __name__ == "__main__":

print("B")

If another Python program executes:

import test

what will normally be printed?

A)A and B

B)Nothing

C)B

D)A

Q50. What will happen?

def fun(n):

if n == 0:

return 1

return n * fun(n)

print(fun(3))

A)3

B)TypeError

C)RecursionError

D)6



Q33. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

print(fun(3))

- A)3
- B)6
- C)None
- D)5

Q34. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

print(result)

- A)0
- B)10
- C)9
- D)24

Q35. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)Nothing
- B)e
- C)ValueError
- D)Wrong Value

Q36. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

- A)30
- 30
- B)30
- 20
- C)20
- 30
- D)20
- 10

Q37. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)25
- B)5
- C)5.0
- D)sqrt(25)

Q38. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A>Error
- B)15
- C)10
- D)5

**Q39. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)B
- B)Nothing
- C)A
- D)A followed by B

Q40. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:
```



```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")
```

- A)B
- B)Nothing
- C)A B
- D)A

Q22. What will be printed?

```
x = 10
```

```
def outer():
    x = 20

    def inner():
        print(x)
```

```
    x = 30
    inner()
```

```
outer()
```

- A)20
- B)10
- C)30
- D)NameError

Q23. What will happen?

```
def outer():
    x = 10

    def inner():
        nonlocal y
        y = 20

    inner()
```

```
outer()
```

- A)20
- B)10
- C)SyntaxError
- D)NameError

Q24. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

- A)20
- B)TypeError
- C)30
- D)SyntaxError

Q25. What will normally happen?

```
import math
import math
```

- A)The module is normally loaded once and reused
- B)ImportError
- C)The module is normally loaded and executed twice
- D)SyntaxError

Q26. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)25
- B)TypeError
- C)SyntaxError
- D)15

Q27. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
```

- A)A
- B)B
- C)A B
- D)SyntaxError

Q28. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
```



```
def first():
    second()

def second():
    third()

def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)NameError
B)TypeError
C)Handled
D)RuntimeError cannot propagate through functions

Q9. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)20
B)None
C)NameError
D)10

Q10. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

A)Error
B)Only Hello

C)x
display
D)10
Hello

Q11. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal x
        x = x + 5
        return x
```

```
    inner()
    return x
```

```
print(outer())
```

A)10
B)5
C)15
D)UnboundLocalError

Q12. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
A>Error
B>Hello None
C)None
D>Hello
```

Q13. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

A)A, followed by propagation of the same exception
B)B
C)A, and execution ends normally
D)No Output



```
print(type(e).__name__)
```

- A)TypeError
- B)MyError
- C)Something went wrong
- D)Exception

Q46. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B

B)A C

C)A

D)B C

Q47. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)random cannot be imported
- B)The local file can shadow the intended standard-library module
- C)Python automatically renames the local file
- D)Python will always reject it

Q48. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)No Output

B)B

C)A, followed by propagation of the same exception

D)A, and execution ends normally

Q49. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
    raise ValueError("negative")
```

```
    return x
```

```
except ValueError:
```

```
    return 100
```

```
finally:
```

```
    print("cleanup")
```

```
print(fun(-5))
```

A)cleanup

100

B)negative

cleanup

C)cleanup

-5

D)100

cleanup

Q50. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)Finally followed by 10

B)Finally only

C)10 followed by Finally

D)10 only



Q33. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        return x + 5  
  
    return inner
```

```
f = outer()  
print(f())  
A)15  
B)None  
C)NameError  
D)10
```

Q34. What will happen?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal y  
        y = 20  
  
    inner()
```

```
outer()  
A)NameError  
B)20  
C)10  
D)SyntaxError
```

Q35. What will be printed?

```
from functools import reduce  
  
numbers = [1, 2, 3, 4]  
  
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)  
  
print(result)  
A)24  
B)10  
C)0  
D)9
```

Q36. What will be printed?

```
def fun(a, b):  
    return a + b  
  
print(fun(10, b=20))  
A)20  
B)SyntaxError  
C)30  
D)TypeError
```

Q37. Which object keeps track of modules that have already been imported during the current Python process?

```
A)sys.imports  
B).__module__  
C)sys.modules  
D)sys.path
```

Q38. What will be printed?

```
x = 10  
  
f = lambda: x  
  
x = 20  
  
print(f())  
A)10  
B)NameError  
C)None  
D)20
```

Q39. Suppose sample.py contains:

```
x = 10  
  
def display():  
    print("Hello")
```

What will this print?

```
import sample  
  
print(sample.x)  
sample.display()  
A)10  
Hello  
B)Only Hello
```



D)Both import only sqrt

Q21. if test.py has
print("A")

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

- A)Nothing
- B)A
- C)A followed by B
- D)B

Q22. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

- A)3
- B)RecursionError
- C)6
- D)TypeError

Q23. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

- A)[10]
- [10, 10]
- B)[10, 10]
- [10]
- C)Error
- D)[10]
- [10]

Q24. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(
```

```
lambda x: x * 2,  
    filter(lambda x: x % 2 == 0,  
numbers)  
    )  
)
```

print(result)

- A)[2, 4, 6, 8, 10]
- B)[1, 3, 5]
- C)[2, 4]
- D)[4, 8]

Q25. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

print(outer())

- A)10
- B)5
- C)UnboundLocalError
- D)15

Q26. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```



print(fun(30))

- A)10
- B)-10
- C)20
- D)30

Q8. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)20
- B)UnboundLocalError
- C)NameError
- D)10

Q9. What happens here?

```
def fun(a):  
    return a
```

print(fun())

- A)0
- B)NameError
- C)None
- D)TypeError

Q10. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

- A)InvalidAgeError
- 15

- B)Nothing, because custom exceptions cannot be caught
- C)InvalidAgeError
- Age is too low
- D)Exception
- Age is too low

Q11. What will be the output?

import math

print(math.sqrt(25))

- A)5
- B)25
- C)sqrt(25)
- D)5.0

Q12. What will be printed?

x = 10

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

outer()

- A)30
- B)20
- C)10
- D)NameError

Q13. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)20
- B)10
- C)None
- D)RuntimeError

Q14. What will be printed?



B)15 15

C)5 5

D)5 15

Q45. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

A)10

B)UnboundLocalError

C)5

D)15

Q46. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.imports

B)sys.path

C)__module__

D)sys.modules

Q47. What happens?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, a=20))
```

A)20

B)TypeError

C)30

D)SyntaxError

Q48. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
print("C")
```

A)A

B)B

C)B C

D)A C

Q49. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded once and reused

B)The module is normally loaded and executed twice

C)ImportError

D)SyntaxError

Q50. if test.py has

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```

If test.py is executed directly, what will be printed?

A)A

B)A followed by B

C)Nothing

D)B



return inner

**f = outer()
print(f())**

- A)15
- B)None
- C)10
- D)NameError

Q30. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)20
- B)10
- C)RuntimeError
- D)None

Q31. What happens here?

from math import sqrt

print(sqrt(36))

- A)6
- B)6.0
- C)NameError
- D)36

Q32. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)cleanup
100
- B)negative
cleanup

- C)100
cleanup
- D)cleanup
-5

Q33. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

print(fun())

- A)10 only
- B)Finally only
- C)10 followed by Finally
- D)Finally followed by 10

Q34. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

- A)TypeError
- B)3
- C)RecursionError
- D)6

Q35. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

- A)NameError
- B)20
- C)None
- D)10

Q36. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```



```
try:
    first()
except RuntimeError:
    print("Handled")
A)Handled
B)TypeError
C)NameError
D)RuntimeError cannot propagate through functions
```

Q19. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)Both import only sqrt
- B)First imports only sqrt
- C)There is no difference
- D)First provides the module namespace; second imports sqrt directly

Q20. What will be printed?

```
def make_counter():
    count = 0
```

```
    def counter():
        nonlocal count
        count += 1
        return count
```

```
    return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

- A)1
- 2
- 1
- 3
- B)1
- 1
- 1

- 1
- C)1
- 2
- 3
- 4
- D)0
- 1
- 0
- 2

Q21. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

- A)10
- B)UnboundLocalError
- C)20
- D)NameError

Q22. What will be printed?

```
def fun():
    print("A")
    return
    print("B")
```

```
fun()
```

- A)A
- B)B
- C)Nothing
- D)A B

Q23. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
A>Error
B)[10]
[10]
```



```
print(fun(b=5, a=20))
```

- A)15
- B)25
- C)SyntaxError
- D)TypeError

Q6. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

```
print(result)
```

- A)[2, 4, 6, 8, 10]
- B)[4, 8]
- C)[1, 3, 5]
- D)[2, 4]

Q7. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)B
- B)A, and execution ends normally
- C)No Output
- D)A, followed by propagation of the same exception

Q8. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)None
- B>Error
- C>Hello None
- D>Hello

Q9. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

- A)x
- display
- B)10
- Hello
- C>Error
- D)Only Hello

Q10. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A)TypeError
- B)20
- C)SyntaxError
- D)30

Q11. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)UnboundLocalError
- B)NameError
- C)10
- D)50

Q12. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```



```

result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)

```

print(result)

A)[2, 4, 6, 8, 10]

B)[2, 4]

C)[4, 8]

D)[1, 3, 5]

Q49. What is the output?

```

def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")

```

print(fun())

A)A

B

10

D

B)A

C

D

20

C)A

B

C

D

10

D)A

B

D

10

Q50. What will be printed?

```

class InvalidAgeError(Exception):
    pass

```

```

def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"

```

```

try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)

```

A)Nothing, because custom exceptions cannot be caught

B)InvalidAgeError

15

C)InvalidAgeError

Age is too low

D)Exception

Age is too low



fun()

- A)10
- B)UnboundLocalError
- C)NameError
- D)50

Q36. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)A and B
- B)Nothing
- C)B
- D)A

Q37. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

- A)15
- B)UnboundLocalError
- C)10
- D)5

Q38. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
print("C")
```

- A)A C
- B)B C
- C)A
- D)B

Q39. What will be the output?

```
x = 10
```

```
def fun():
```

```
    print(x)
```

```
    x = 20
```

```
fun()
```

- A)20

- B)10

- C)NameError

- D)UnboundLocalError

Q40. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)10

- 10

- B)ImportError

- C)Module loaded

- 10

- D)Module loaded

- Module loaded

- 10

Q41. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```



return a

print(fun())

- A)0
- B)NameError
- C)None
- D)TypeError

Q23. Why is

try:

print(10 / 0)

except Exception:

print("A")

except ZeroDivisionError:

print("B")

codes's second handler never reached?

- A)except blocks execute randomly
- B)ZeroDivisionError occurs before Exception
- C)Exception is a superclass of ZeroDivisionError
- D)ZeroDivisionError is not an exception

Q24. What will be the output?

x = 10

def outer():

x = 20

def inner():

global x

x = 30

inner()

print(x)

outer()

print(x)

- A)30
- 30
- B)20
- 30
- C)20
- 10
- D)30
- 20

Q25. Suppose sample.py contains:

x = 10

def display():

print("Hello")

What will this print?

import sample

print(sample.x)

sample.display()

- A)Error
- B)x
- display
- C)10
- Hello
- D)Only Hello

Q26. What is the output?

def outer():

x = 10

def inner():

return x + 5

return inner

f = outer()

print(f())

- A)None
- B)NameError
- C)15
- D)10

Q27. What will happen?

def outer():

x = 10

def inner():

nonlocal y

y = 20

inner()

outer()

- A)NameError
- B)20
- C)SyntaxError



from math import sqrt

- A)Both import only sqrt
- B)There is no difference
- C)First imports only sqrt
- D)First provides the module namespace; second imports sqrt directly

Q9. What does the following function return?

```
def fun():  
    print("Hello")
```

x = fun()

print(x)

- A)None
- B)Hello None
- C)Hello
- D>Error

Q10. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

- A)The local file can shadow the intended standard-library module
- B)Python will always reject it
- C)Python automatically renames the local file
- D)random cannot be imported

Q11. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

- A)6
- B)3
- C)RecursionError
- D)TypeError

Q12. What will be printed?

x = 10

def outer():

x = 20

```
def inner():  
    print(x)
```

**x = 30
inner()**

outer()

- A)30
- B)20
- C)NameError
- D)10

Q13. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

- A)NameError
- B)Handled
- C)TypeError
- D)RuntimeError cannot propagate through functions

Q14. What will normally happen?

**import math
import math**

- A)The module is normally loaded once and reused
- B)The module is normally loaded and executed twice
- C)ImportError
- D)SyntaxError

Q15. What will be the output?

```
def fun(x):  
    try:
```



```
x += 10
return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

- A) 5 5
- B) 5 15
- C) 15 15
- D) 15 5

Q46. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A) Python automatically renames the local file
- B) random cannot be imported
- C) Python will always reject it
- D) The local file can shadow the intended standard-library module

Q47. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A) 10
- B) 20
- C) None
- D) RuntimeError

Q48. What is the output?

```
def outer():
    x = 10

    def inner():
        nonlocal x
        x = x + 5
        return x

    inner()
    return x
```

```
print(outer())
```

- A) 10
- B) 15
- C) 5
- D) UnboundLocalError

Q49. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A) 20
- B) 10
- C) 30
- D) -10

Q50. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

- A) None
- B) 0
- C) NameError
- D) TypeError



Q32. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)5
- B)sqrt(25)
- C)5.0
- D)25

Q33. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)TypeError
- B)15
- C)SyntaxError
- D)25

Q34. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

- A)Only Hello
- B>Error
- C)x
display
- D)10
Hello

Q35. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()
```

- A)10
- B)SyntaxError
- C)NameError
- D)20

Q36. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)NameError
- B)None
- C)20
- D)10

Q37. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

- A)A B
- B)Nothing
- C)A
- D)B

Q38. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)ImportError



- A)10
- B)NameError
- C)15
- D)None

Q19. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

- A)SyntaxError
- B)30
- C)TypeError
- D)20

Q20. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A)SyntaxError
- B)TypeError
- C)20
- D)30

Q21. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)15
- B)5
- C>Error
- D)10

Q22. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)e
- B)ValueError
- C)Wrong Value
- D)Nothing

Q23. What will be printed?

```
def make_counter():
    count = 0
```

```
def counter():
    nonlocal count
    count += 1
    return count
```

```
    return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)0

1

0

2

B)1

2

1

3

C)1

1

1

1

D)1

2

3

4

**Q24. if test.py has
print("A")**

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A
- B)A followed by B
- C)B
- D)Nothing

Q25. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```



print(a(5))

- A)10
- B)func(5)
- C)5
- D)Error

Q8. What is the purpose of this statement?

if __name__ == "__main__":

- A)It identifies code intended to run when the file is executed directly
- B)It prevents syntax errors
- C)It prevents all imports
- D)It creates the main function

Q9. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)cleanup
- 100
- B)negative
- cleanup
- C)100
- cleanup
- D)cleanup
- 5

Q10. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)B
- B)A
- C)A B
- D)Nothing

Q11. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

- A)A
- B
- D
- 10
- B)A
- C
- D
- 20
- C)A
- B
- C
- D
- 10
- D)A
- B
- 10
- D

Q12. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)B C
- B)A
- C)A C
- D)B



D)Module loaded

10

Q46. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)NameError

B)UnboundLocalError

C)50

D)10

Q47. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q48. What is the output?

f = lambda x: x + 5

print(f(10))

A)5

B>Error

C)10

D)15

Q49. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

A)NameError

B)None

C)10

D)20

Q50. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

A)TypeError

B)30

C)SyntaxError

D)20



```
f = outer()
```

```
print(f())
```

A)NameError

B)None

C)10

D)15

Q34. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)A B

B)A

C)SyntaxError

D)B

Q35. What happens when this code runs?

```
def first():
```

```
    second()
```

```
def second():
```

```
    third()
```

```
def third():
```

```
    raise RuntimeError("Failure")
```

```
try:
```

```
    first()
```

```
except RuntimeError:
```

```
    print("Handled")
```

A)RuntimeError cannot propagate through functions

B)Handled

C)TypeError

D)NameError

Q36. What will happen?

```
def fun(a, b=10):
```

```
    return a + b
```

```
print(fun(b=5, a=20))
```

A)15

B)25

C)SyntaxError

D)TypeError

Q37. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)0

B)NameError

C)TypeError

D)None

Q38. What is the output?

```
def fun():
```

```
    try:
```

```
        print("A")
```

```
        raise ValueError
```

```
    except ValueError:
```

```
        print("B")
```

```
        return 10
```

```
    else:
```

```
        print("C")
```

```
        return 20
```

```
    finally:
```

```
        print("D")
```

```
print(fun())
```

A)A

C

D

20

B)A

B

D

10

C)A

B

10

D

D)A

B

C

D

10

Q39. Which object keeps track of modules that have already been



print(x)

A)30

20

B)20

30

C)30

30

D)20

10

Q21. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

print(result)

A)[2, 4, 6, 8, 10]

B)[4, 8]

C)[2, 4]

D)[1, 3, 5]

Q22. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

A)5

B)func(5)

C>Error

D)10

Q23. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
        return x
    except ValueError:
        return 100
    finally:
        print("cleanup")
```

print(fun(-5))

A)100

cleanup

B)cleanup

100

C)cleanup

-5

D)negative

cleanup

Q24. What is the major difference between the following?

import math

and

from math import sqrt

A)There is no difference

B)Both import only sqrt

C)First provides the module namespace; second imports sqrt directly

D)First imports only sqrt

Q25. What will be printed?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        print(x)
```

```
    x = 30
    inner()
```

```
outer()
```

A)20

B)NameError

C)30

D)10

Q26. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

A)20



B)It executes correctly

C)SyntaxError

D)NameError

Q6. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

A)Nothing

B)A

C)B

D)A B

Q7. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

try:

```
    test()
```

except MyError as e:

```
    print(type(e).__name__)
```

A)Exception

B)TypeError

C)Something went wrong

D)MyError

Q8. What happens here?

try:

```
    raise ValueError("Wrong value")
```

except ValueError as e:

```
    print(e)
```

A)Nothing

B)e

C)ValueError

D)Wrong Value

Q9. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:
```

return 20

print(fun())

A)20

B)None

C)RuntimeError

D)10

Q10. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

**codes's second handler never
reached?**

A)ZeroDivisionError is not an exception

B)ZeroDivisionError occurs before
Exception

C)Exception is a superclass of
ZeroDivisionError

D)except blocks execute randomly

**Q11. What is the purpose of this
statement?**

```
if __name__ == "__main__":
```

A)It identifies code intended to run when
the file is executed directly

B)It creates the main function

C)It prevents all imports

D)It prevents syntax errors

Q12. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

print(fun(3))

A)5

B)6

C)3

D)None

Q13. What is the output?

try:

```
    x = int("abc")
```

except ValueError:



print(fun(b=5, a=20))

- A)15
- B)25
- C)SyntaxError
- D)TypeError

Q45. What happens here?

from math import sqrt

print(sqrt(36))

- A)6.0
- B)6
- C)NameError
- D)36

Q46. What will be the output?

import math

print(math.sqrt(25))

- A)5.0
- B)5
- C)25
- D)sqrt(25)

Q47. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

- A)The local file can shadow the intended standard-library module
- B)Python will always reject it
- C)random cannot be imported
- D)Python automatically renames the local file

Q48. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

- A)UnboundLocalError
- B)10
- C)50
- D)NameError

Q49. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)NameError
- B)20
- C)10
- D)UnboundLocalError

Q50. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)Nothing
- B)A
- C)A B
- D)B



print(x)

**outer()
print(x)**

A)20

10

B)30

30

C)20

30

D)30

20

Q33. Suppose module_a.py contains:

print("Module loaded")

value = 10

And another file contains:

**import module_a
import module_a**

print(module_a.value)

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

10

B)10

10

C)ImportError

D)Module loaded

Module loaded

10

Q34. What happens when this code runs?

**def first():
 second()**

**def second():
 third()**

**def third():
 raise RuntimeError("Failure")**

try:

first()

except RuntimeError:

print("Handled")

A)TypeError

B)Handled

C)NameError

D)RuntimeError cannot propagate through functions

Q35. What will be the output?

def fun(x):

try:

if x < 0:

raise ValueError("negative")

return x

except ValueError:

return 100

finally:

print("cleanup")

print(fun(-5))

A)cleanup

100

B)negative

cleanup

C)100

cleanup

D)cleanup

-5

Q36. What will be printed?

def fun(x=[]):

x.append(10)

return x

a = fun()

b = fun()

print(a)

print(b)

A)[10]

[10]

B)[10, 10]

[10]

C)Error

D)[10]

[10, 10]



```
def inner():  
    nonlocal x  
    x = x + 5  
    return x
```

```
inner()  
return x
```

```
print(outer())
```

- A)5
- B)UnboundLocalError
- C)15
- D)10

Q21. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

- A)InvalidAgeError
- 15
- B)InvalidAgeError
- Age is too low
- C)Nothing, because custom exceptions cannot be caught
- D)Exception
- Age is too low

Q22. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)10
- B)15
- C)5
- D>Error

Q23. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

- A)First imports only sqrt
- B)Both import only sqrt
- C)There is no difference
- D)First provides the module namespace; second imports sqrt directly

Q24. What will normally happen?

```
import math  
import math
```

- A)The module is normally loaded and executed twice
- B)ImportError
- C)SyntaxError
- D)The module is normally loaded once and reused

Q25. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

```
print(fun())
```

- A)A
- B
- D
- 10
- B)A
- C
- D
- 20



imported during the current Python process?

- A)sys.path
- B)sys.imports
- C)sys.modules
- D)__module__

Q8. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

print(fun(3))

- A)None
- B)5
- C)6
- D)3

Q9. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

print(x, y)

- A)15 5
- B)5 5
- C)15 15
- D)5 15

Q10. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents syntax errors
- B)It identifies code intended to run when the file is executed directly
- C)It prevents all imports
- D)It creates the main function

Q11. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went
```

wrong")

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

- A)TypeError
- B)Something went wrong
- C)Exception
- D)MyError

Q12. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

print(fun())

- A)10 followed by Finally
- B)Finally followed by 10
- C)Finally only
- D)10 only

Q13. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

print(result)

- A)[2, 4]
- B)[1, 3, 5]
- C)[2, 4, 6, 8, 10]
- D)[4, 8]

Q14. What happens here?

```
def fun(a):  
    return a
```

print(fun())

- A)TypeError
- B)0



Q46. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A) It prevents all imports
- B) It prevents syntax errors
- C) It identifies code intended to run when the file is executed directly
- D) It creates the main function

Q47. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

A) 0

1

0

2

B) 1

2

1

3

C) 1

1

1

1

D) 1

2

3

4

Q48. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A) SyntaxError

B) 20

C) TypeError

D) 30

Q49. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A) A, followed by propagation of the same exception

B) B

C) No Output

D) A, and execution ends normally

Q50. What will normally happen?

```
import math
```

```
import math
```

A) The module is normally loaded once and reused

B) SyntaxError

C) ImportError

D) The module is normally loaded and executed twice



```

if n == 0:
    return 1
return n * fun(n)

```

print(fun(3))

- A)TypeError
- B)6
- C)RecursionError
- D)3

Q35. What happens here?

```

def fun(a):
    return a

```

print(fun())

- A)TypeError
- B)NameError
- C)None
- D)0

Q36. What is the output?

```

try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")

```

- A)SyntaxError
- B)A B
- C)B
- D)A

Q37. What happens when this code runs?

```

def first():
    second()

def second():
    third()

def third():
    raise RuntimeError("Failure")

try:
    first()
except RuntimeError:
    print("Handled")

```

A)RuntimeError cannot propagate through functions

- B)NameError
- C)TypeError
- D)Handled

Q38. What will be printed?

```

def fun():
    try:
        return 10
    finally:
        print("Finally")

```

print(fun())

- A)10 followed by Finally
- B)Finally followed by 10
- C)10 only
- D)Finally only

Q39. What is the output?

```

def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")

```

print(fun())

- A)A
- C
- D
- 20
- B)A
- B
- C
- D
- 10
- C)A
- B
- 10
- D



print("C")

- A)A C
- B)A
- C)B C
- D)B

Q21. What will be the output?

import math

print(math.sqrt(25))

- A)5
- B)5.0
- C)25
- D)sqrt(25)

Q22. What will happen?

```
def fun(a, b=10):  
    return a + b
```

print(fun(b=5, a=20))

- A)TypeError
- B)15
- C)25
- D)SyntaxError

Q23. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)20
- B)10
- C)NameError
- D)UnboundLocalError

Q24. What does the following function return?

```
def fun():  
    print("Hello")
```

x = fun()

```
print(x)  
A)Hello  
B>Error
```

C)Hello None

D)None

Q25. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
            raise ValueError("negative")
```

```
        return x
```

```
    except ValueError:
```

```
        return 100
```

```
    finally:
```

```
        print("cleanup")
```

print(fun(-5))

- A)100
- cleanup
- B)negative
- cleanup
- C)cleanup
- 100
- D)cleanup
- 5

Q26. What is the output?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        return 20
```

print(fun())

- A)20
- B)None
- C)RuntimeError
- D)10

Q27. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

fun()

- A)A
- B)B
- C)Nothing
- D)A B



Q7. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)10
- B)Error
- C)15
- D)5

Q8. if test.py has

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A
- B)B
- C)A followed by B
- D)Nothing

Q9. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)A
- B)B
- C)A C
- D)B C

Q10. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[1, 3, 5]

- B)[2, 4]
- C)[4, 8]
- D)[2, 4, 6, 8, 10]

Q11. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

- A)9
- B)0
- C)10
- D)24

Q12. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        return x + 5  
  
    return inner
```

```
f = outer()  
print(f())
```

- A)None
- B)NameError
- C)10
- D)15

Q13. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)e
- B)Nothing
- C)Wrong Value
- D)ValueError

Q14. Suppose sample.py contains:

```
x = 10
```



print(fun(3))

- A)5
- B)6
- C)3
- D)None

Q46. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

- A)A B
- B)Nothing
- C)B
- D)A

Q47. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
        x = x + 5  
        return x  
  
    inner()  
    return x
```

print(outer())

- A)5
- B)15
- C)UnboundLocalError
- D)10

Q48. Suppose sample.py contains:

x = 10

```
def display():  
    print("Hello")
```

What will this print?

```
import sample  
  
print(sample.x)
```

sample.display()

- A)Error
- B)x
- display
- C)10
- Hello
- D)Only Hello

Q49. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

- A)The local file can shadow the intended standard-library module
- B)Python automatically renames the local file
- C)Python will always reject it
- D)random cannot be imported

Q50. What happens?

```
def fun(a, b):  
    return a + b
```

print(fun(10, a=20))

- A)TypeError
- B)30
- C)SyntaxError
- D)20

- A)B C
- B)B
- C)A C
- D)A

Q33. What will be printed?

```
def make_counter():
    count = 0
```

```
    def counter():
        nonlocal count
        count += 1
        return count
```

```
    return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)0

1

0

2

B)1

1

1

1

C)1

2

1

3

D)1

2

3

4

Q34. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

A)Wrong Value

B)ValueError

C)e

D)Nothing

Q35. What will be the output?

```
x = 10
```

```
def fun():
    print(x)
    x = 20
```

```
fun()
```

A)UnboundLocalError

B)10

C)NameError

D)20

Q36. What will happen?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal y
        y = 20
```

```
    inner()
```

```
outer()
```

A)20

B)10

C)SyntaxError

D)NameError

Q37. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

```
print(result)
```

A)[2, 4, 6, 8, 10]

B)[1, 3, 5]

C)[4, 8]

D)[2, 4]



```
def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

Age is too low

D)InvalidAgeError

15

Q20. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)Handled

B)RuntimeError cannot propagate through functions

C)TypeError

D)NameError

Q21. What will be printed?

```
x = 10
```

```
def outer():
    x = 20
```

```
def inner():
    print(x)
```

```
x = 30
inner()
```

```
outer()
```

A)10

B)20

C)NameError

D)30

Q22. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)10

B)50

C)UnboundLocalError

D)NameError

Q23. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A)First imports only sqrt

B)First provides the module namespace; second imports sqrt directly

C)There is no difference

D)Both import only sqrt

Q24. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

A)SyntaxError

B)It executes correctly

C)TypeError

D)NameError



D)30

30

Q6. What is the output?

try:

```
x = int("abc")
```

except ValueError:

```
print("A")
```

else:

```
print("B")
```

finally:

```
print("C")
```

A)A C

B)B C

C)A

D)B

Q7. What will be printed?

try:

```
print(10 / 0)
```

except ZeroDivisionError:

```
print("A")
```

except Exception:

```
print("B")
```

A)A B

B)A

C)Nothing

D)B

Q8. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)10

B)15

C)None

D)NameError

Q9. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents all imports

B)It creates the main function

C)It prevents syntax errors

D)It identifies code intended to run when the file is executed directly

Q10. What is the output?

try:

```
print(10 / 0)
```

except Exception:

```
print("A")
```

except ZeroDivisionError:

```
print("B")
```

A)B

B)A

C)SyntaxError

D)A B

Q11. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)36

B)6

C)NameError

D)6.0

Q12. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A)5

B)10

C)func(5)

D>Error

Q13. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)20

B)None



b = fun()

print(a)

print(b)

A)[10]

[10, 10]

B)[10]

[10]

C)[10, 10]

[10]

D)Error

Q46. What will happen?

def fun(a, b=10):

return a + b

print(fun(b=5, a=20))

A)SyntaxError

B)15

C)TypeError

D)25

Q47. What will happen?

def outer():

x = 10

def inner():

nonlocal y

y = 20

inner()

outer()

A)SyntaxError

B)10

C)NameError

D)20

Q48. What will be printed?

def fun():

try:

return 10

finally:

print("Finally")

print(fun())

A)10 followed by Finally

B)Finally followed by 10

C)10 only

D)Finally only

Q49. What is the output?

def fun():

try:

return 10

finally:

return 20

print(fun())

A)RuntimeError

B)None

C)20

D)10

Q50. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)The local file can shadow the intended standard-library module

C)Python will always reject it

D)Python automatically renames the local file



D)InvalidAgeError

15

Q32. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

A)NameError

B)RuntimeError cannot propagate through functions

C)TypeError

D)Handled

Q33. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)TypeError

B)SyntaxError

C)20

D)30

Q34. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)NameError

B)None

C)10

D)20

Q35. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B C

C)A C

D)B

Q36. What will normally happen?

```
import math
```

```
import math
```

A)SyntaxError

B)ImportError

C)The module is normally loaded once and reused

D)The module is normally loaded and executed twice

Q37. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

A)NameError

B)TypeError

C)It executes correctly

D)SyntaxError

Q38. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)10

B)NameError

C)15

D)None



Q19. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)30
- B)20
- C)10
- D)-10

Q20. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)First imports only sqrt
- B)There is no difference
- C)First provides the module namespace; second imports sqrt directly
- D)Both import only sqrt

Q21. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)NameError
- B)UnboundLocalError
- C)50
- D)10

Q22. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

- A)SyntaxError
- B)B
- C)A B
- D)A

Q23. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[2, 4]
- B)[1, 3, 5]
- C)[2, 4, 6, 8, 10]
- D)[4, 8]

Q24. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)
```

- A)15 5
- B)15 15
- C)5 5
- D)5 15

Q25. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

- A)No Output
- B)A, followed by propagation of the same exception
- C)B
- D)A, and execution ends normally

Q26. if test.py has

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```



```
test()
except MyError as e:
    print(type(e).__name__)
```

A)MyError

B)TypeError

C)Exception

D)Something went wrong

Q6. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

A)5

B)10

C>Error

D)func(5)

Q7. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6

B)36

C)NameError

D)6.0

Q8. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

A)x

display

B)Only Hello

C>Error

D)10

Hello

Q9. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)5

B)6

C)3

D)None

Q10. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)24

B)10

C)0

D)9

Q11. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

A)0

B)TypeError

C)NameError

D)None

Q12. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?



second()

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")  
A)NameError  
B)Handled  
C)RuntimeError cannot propagate through  
functions  
D)TypeError
```

Q47. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)25
B)5
C)sqrt(25)
D)5.0

Q48. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

A)20
30
B)20
10
C)30
30

D)30
20

Q49. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

A)A
B)Nothing
C)A B
D)B

Q50. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```

A)10 only
B)Finally only
C)Finally followed by 10
D)10 followed by Finally



```
return
return x + fun(x - 1)
```

print(fun(3))

- A)6
- B)3
- C)None
- D)5

Q36. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

print(fun())

- A)A
- C
- D
- 20
- B)A
- B
- D
- 10
- C)A
- B
- C
- D
- 10
- D)A
- B
- 10
- D

Q37. What will be printed?

```
def make_counter():
    count = 0

    def counter():
```

```
nonlocal count
count += 1
return count
```

return counter

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

- A)1
- 2
- 1
- 3
- B)0
- 1
- 0
- 2
- C)1
- 1
- 1
- 1
- D)1
- 2
- 3
- 4

Q38. What will be printed?

```
def fun(a, b):
    return a + b
```

print(fun(10, b=20))

- A)20
- B)30
- C)TypeError
- D)SyntaxError

Q39. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
        return x
    except ValueError:
        return 100
```



Q22. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)  
A)[10, 10]  
[10]  
B)Error  
C)[10]  
[10, 10]  
D)[10]  
[10]
```

Q23. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()  
A)Error  
B)Only Hello  
C)x  
display  
D)10  
Hello
```

Q24. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))  
A)20  
B)-10  
C)30  
D)10
```

Q25. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        return x + 5  
  
    return inner
```

```
f = outer()  
print(f())  
A)NameError  
B)10  
C)None  
D)15
```

Q26. What will happen?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal y  
        y = 20  
  
    inner()
```

```
outer()  
A)SyntaxError  
B)20  
C)10  
D)NameError
```

Q27. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

```
A)B C  
B)A C  
C)A  
D)B
```

Q28. What is the output?



A)15 15

B)5 5

C)15 5

D)5 15

Q8. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)24

B)0

C)9

D)10

Q9. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)random cannot be imported

B)Python will always reject it

C)Python automatically renames the local file

D)The local file can shadow the intended standard-library module

Q10. What will be printed?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

```
outer()
```

A)30

B)NameError

C)10

D)20

Q11. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)TypeError

B)RecursionError

C)6

D)3

Q12. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)10

B)5

C)15

D>Error

Q13. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

A)NameError

B)SyntaxError

C)It executes correctly

D)TypeError

Q14. if test.py has print("A")

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

A)Nothing

B)A

C)B

D)A followed by B

Q15. What happens here?

```
def fun(a):  
    return a
```



```

    print("A")
    raise ValueError
except ValueError:
    print("B")
    return 10
else:
    print("C")
    return 20
finally:
    print("D")

```

print(fun())

- A)A
- B
- D
- 10
- B)A
- B
- 10
- D
- C)A
- C
- D
- 20
- D)A
- B
- C
- D
- 10

Q47. What will happen?

```

def fun(a, b=10):
    return a + b

```

print(fun(b=5, a=20))

- A)15
- B)TypeError
- C)25
- D)SyntaxError

Q48. What is the output?

```

try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")

```

```

finally:
    print("C")

```

- A)B C
- B)B
- C)A
- D)A C

Q49. What is the output?

```

try:
    raise ValueError
except ValueError:
    print("A")
    raise

```

- A)B
- B)A, and execution ends normally
- C)No Output
- D)A, followed by propagation of the same exception

Q50. What will happen?

```

def fun(n):
    if n == 0:
        return 1
    return n * fun(n)

```

print(fun(3))

- A)6
- B)3
- C)TypeError
- D)RecursionError



Q34. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)NameError

B)10

C)20

D)30

Q35. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)15

B)None

C)10

D)NameError

Q36. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)It executes correctly

B)SyntaxError

C)NameError

D)TypeError

Q37. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)Python will always reject it

C)Python automatically renames the local file

D)The local file can shadow the intended standard-library module

Q38. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)0

B)None

C)NameError

D)TypeError

Q39. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B

C)B C

D)A C

Q40. What will be the output?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        global x
```

```
        x = 30
```

```
    inner()
```

```
    print(x)
```

```
outer()
```

```
print(x)
```



```
a = fun
print(a(5))
```

- A)func(5)
- B)5
- C)Error
- D)10

Q22. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Hello None
- B)Error
- C)Hello
- D)None

Q23. What will be printed?

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")
```

- A)B
- B)A B
- C)A
- D)Nothing

Q24. What will happen?

```
def outer():
    x = 10

    def inner():
        nonlocal y
        y = 20
```

```
    inner()
```

```
outer()
```

- A)20
- B)10
- C)NameError
- D)SyntaxError

Q25. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A)30
- B)-10
- C)10
- D)20

Q26. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A)20
- B)30
- C)TypeError
- D)SyntaxError

Q27. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)A
- B)Nothing
- C)A and B
- D)B

Q28. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)10 only
- B)Finally only
- C)Finally followed by 10



Q8. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
        x = x + 5  
        return x  
  
    inner()  
    return x
```

print(outer())

- A)10
- B)UnboundLocalError
- C)15
- D)5

Q9. What will normally happen?

```
import math  
import math
```

- A)ImportError
- B)The module is normally loaded once and reused
- C)SyntaxError
- D)The module is normally loaded and executed twice

Q10. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.path
- B)sys.imports
- C)sys.modules
- D).__module__

Q11. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

print(fun(3))

- A)6
- B)None
- C)3
- D)5

Q12. What will be printed?

```
class InvalidAgeError(Exception):  
    pass  
  
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is too low")  
    return "Allowed"
```

try:

```
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

- A)Exception
- Age is too low
- B)Nothing, because custom exceptions cannot be caught
- C)InvalidAgeError
- Age is too low
- D)InvalidAgeError
- 15

Q13. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

try:

```
    first()  
except RuntimeError:  
    print("Handled")
```

- A)NameError
- B)Handled
- C)RuntimeError cannot propagate through functions
- D)TypeError



C)20

30

D)30

20

Q47. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)None

B)10

C)NameError

D)20

Q48. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

1

3

B)1

2

3

4

C)0

1

0

2

D)1

1

1

1

Q49. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)10 only

B)10 followed by Finally

C)Finally followed by 10

D)Finally only

Q50. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)20

C)SyntaxError

D)TypeError



Q34. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents syntax errors
- B)It prevents all imports
- C)It identifies code intended to run when the file is executed directly
- D)It creates the main function

Q35. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

- A)Handled
- B)NameError
- C)TypeError
- D)RuntimeError cannot propagate through functions

Q36. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
- Module loaded

10

B)Module loaded

10

C)ImportError

D)10

10

Q37. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)NameError
- B)TypeError
- C)It executes correctly
- D)SyntaxError

Q38. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

- A)5
- B)15
- C)10
- D)UnboundLocalError

Q39. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)Python will always reject it
- B)The local file can shadow the intended standard-library module
- C)random cannot be imported
- D)Python automatically renames the local file

Q40. What is the output?

```
def fun(x):
```



Q19. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)5
- B)5.0
- C)25
- D)sqrt(25)

Q20. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Hello None
- B)Hello
- C>Error
- D)None

Q21. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)
```

```
print(b)
```

- A)[10, 10]
- [10]
- B)[10]
- [10]
- C)[10]
- [10, 10]
- D>Error

Q22. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

- A)0
- B)TypeError

C)None

D)NameError

Q23. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)SyntaxError

B)A

C)B

D)A B

Q24. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)No Output

B)A, followed by propagation of the same exception

C)B

D)A, and execution ends normally

Q25. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():
```

```
    raise MyError("Something went wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)Exception

B)MyError

C)TypeError

D)Something went wrong

Q26. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```



D)30

Q7. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)10
- B)20
- C)RuntimeError
- D)None

Q8. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)cleanup
- 100
- B)negative
cleanup
- C)cleanup
- 5
- D)100
cleanup

Q9. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:
```

print("D")

print(fun())

- A)A
- B
- D
- 10
- B)A
- B
- 10
- D
- C)A
- B
- C
- D
- 10
- D)A
- C
- D
- 20

**Q10. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)A
- B)A followed by B
- C)B
- D)Nothing

Q11. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:
```



D)10

Q46. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)SyntaxError

B)10

C)20

D)NameError

Q47. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded and executed twice

B)ImportError

C)SyntaxError

D)The module is normally loaded once and reused

Q48. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)A, followed by propagation of the same exception

B)A, and execution ends normally

C)No Output

D)B

Q49. What will be the output?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        global x
```

```
x = 30
```

```
inner()
```

```
print(x)
```

```
outer()
```

```
print(x)
```

A)20

10

B)30

20

C)30

30

D)20

30

**Q50. if test.py has
print("A")**

```
if __name__ == "__main__":
```

```
    print("B")
```

If test.py is executed directly, what will be printed?

A)Nothing

B)A

C)B

D)A followed by B



Q32. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)Only Hello

B)10

Hello

C)Error

D)x

display

Q33. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.path

B)sys.modules

C)__module__

D)sys.imports

Q34. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)SyntaxError

C)20

D)TypeError

Q35. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()
```

```
except MyError as e:  
    print(type(e).__name__)
```

A)Something went wrong

B)Exception

C)MyError

D)TypeError

Q36. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)20

B)10

C)30

D)-10

Q37. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)30

B)20

C)SyntaxError

D)TypeError

Q38. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A)Exception is a superclass of ZeroDivisionError

B)except blocks execute randomly

C)ZeroDivisionError occurs before Exception

D)ZeroDivisionError is not an exception

Q39. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```



normally printed?

A)10

10

B)ImportError

C)Module loaded

10

D)Module loaded

Module loaded

10

Q19. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)It executes correctly

B)TypeError

C)SyntaxError

D)NameError

Q20. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)A B

B)A

C)B

D)SyntaxError

Q21. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
            raise ValueError("negative")
```

```
        return x
```

```
    except ValueError:
```

```
        return 100
```

```
    finally:
```

```
        print("cleanup")
```

```
print(fun(-5))
```

A)100

cleanup

B)cleanup

-5

C)cleanup

100

D)negative

cleanup

Q22. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)B

C)B C

D)A

Q23. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)Finally only

B)10 followed by Finally

C)Finally followed by 10

D)10 only

Q24. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A>Error

B)func(5)

C)10

D)5

Q25. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```



```

raise ValueError
except ValueError:
    print("B")
    return 10
else:
    print("C")
    return 20
finally:
    print("D")

```

print(fun())

A)A

C

D

20

B)A

B

10

D

C)A

B

C

D

10

D)A

B

D

10

Q6. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)The local file can shadow the intended standard-library module

B)random cannot be imported

C)Python automatically renames the local file

D)Python will always reject it

Q7. What happens here?

from math import sqrt

print(sqrt(36))

A)6.0

B)36

C)6

D)NameError

Q8. What is the purpose of this statement?

if __name__ == "__main__":

A)It creates the main function

B)It prevents all imports

C)It identifies code intended to run when the file is executed directly

D)It prevents syntax errors

Q9. What is the output?

def fun(x):

x += 10

return x

x = 5

y = fun(x)

print(x, y)

A)15 5

B)15 15

C)5 5

D)5 15

Q10. What will be printed?

def fun(x=[]):

x.append(10)

return x

a = fun()

b = fun()

print(a)

print(b)

A)[10]

[10]

B)[10]

[10, 10]

C)[10, 10]

[10]

D>Error

Q11. What will be printed?

try:

print(10 / 0)

except ZeroDivisionError:



Q45. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

- A)NameError
- B)UnboundLocalError
- C)10
- D)20

Q46. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It identifies code intended to run when the file is executed directly
- B)It prevents all imports
- C)It creates the main function
- D)It prevents syntax errors

Q47. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

- A)10
- B)UnboundLocalError
- C)5
- D)15

Q48. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)SyntaxError
- B)NameError
- C)It executes correctly

D)TypeError

Q49. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

- A)SyntaxError
- B)A
- C)A B
- D)B

Q50. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

- A)TypeError
- B)MyError
- C)Exception
- D)Something went wrong



```
outer()
print(x)
```

- A)20
- 30
- B)20
- 10
- C)30
- 30
- D)30
- 20

Q32. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A)RuntimeError
- B)None
- C)20
- D)10

Q33. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

- A)[2, 4]
- B)[4, 8]
- C)[2, 4, 6, 8, 10]
- D)[1, 3, 5]

Q34. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)Nothing
- B)B
- C)A
- D)A and B

Q35. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)Finally followed by 10
- B)10 only
- C)Finally only
- D)10 followed by Finally

Q36. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)Python automatically renames the local file
- B)random cannot be imported
- C)The local file can shadow the intended standard-library module
- D)Python will always reject it

Q37. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)NameError
- B)None
- C)20
- D)10

Q38. What will happen?



B)The module is normally loaded and executed twice
C)SyntaxError
D)The module is normally loaded once and reused

Q19. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)NameError
B)36
C)6
D)6.0

Q20. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)30
B)10
C)20
D)-10

Q21. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)TypeError
B)30
C)20
D)SyntaxError

Q22. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

A)1

2

3

4

B)1

2

1

3

C)0

1

0

2

D)1

1

1

1

Q23. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

A)A
B)Nothing
C)A B
D)B

Q24. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

A)25
B)TypeError
C)15
D)SyntaxError

Q25. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```



B)10

Hello

C)x

display

D)Only Hello

Q6. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

A)9

B)24

C)0

D)10

Q7. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)50

B)NameError

C)UnboundLocalError

D)10

Q8. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)ZeroDivisionError is not an exception

B)ZeroDivisionError occurs before Exception

C)except blocks execute randomly

D)Exception is a superclass of ZeroDivisionError

Q9. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)A

C)B C

D)B

Q10. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)B C

B)B

C)A C

D)A

Q11. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)10

B)SyntaxError

C)20

D)NameError

Q12. What happens when this code runs?



C)The local file can shadow the intended standard-library module

D)Python will always reject it

Q46. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)5

B>Error

C)15

D)10

Q47. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)Exception

B)MyError

C)TypeError

D)Something went wrong

Q48. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)6

B)3

C)5

D)None

Q49. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

```
print(fun())
```

A)RuntimeError

B)20

C)10

D)None

Q50. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)x

display

B)Only Hello

C)10

Hello

D>Error



10
D
B)A
C
D
20
C)A
B
D
10
D)A
B
C
D
10

Q34. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

A)[10]
[10]
B)[10, 10]
[10]
C)Error
D)[10]
[10, 10]

**Q35. Why is
try:**

```
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")  
codes's second handler never  
reached?
```

A)ZeroDivisionError is not an exception
B)ZeroDivisionError occurs before
Exception
C)Exception is a superclass of
ZeroDivisionError
D)except blocks execute randomly

Q36. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6
B)36
C)6.0
D)NameError

Q37. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

A)10
B)5
C)15
D)UnboundLocalError

Q38. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

A)[4, 8]
B)[2, 4]
C)[1, 3, 5]
D)[2, 4, 6, 8, 10]

**Q39. if test.py has
print("A")**

```
if __name__ == "__main__":
```



```
def inner():
    global x
    x = 30
```

```
inner()
print(x)
```

```
outer()
print(x)
```

A)30

30

B)30

20

C)20

30

D)20

10

Q22. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

A)SyntaxError

B)TypeError

C)25

D)15

Q23. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

A)10 only

B)10 followed by Finally

C)Finally followed by 10

D)Finally only

Q24. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

A)No Output

B)B

C)A, followed by propagation of the same exception

D)A, and execution ends normally

Q25. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5.0

B)25

C)sqrt(25)

D)5

Q26. What will be printed?

```
def make_counter():
    count = 0
```

```
def counter():
    nonlocal count
    count += 1
    return count
```

```
return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

1

1

1

B)1

2

1

3

C)0

1

0

2

D)1

2

3

4



except ZeroDivisionError:

print("B")

A)SyntaxError

B)A

C)B

D)A B

Q8. What is the output?

def fun(x):

x += 10

return x

x = 5

y = fun(x)

print(x, y)

A)15 5

B)5 15

C)5 5

D)15 15

Q9. What happens when this code runs?

def first():

second()

def second():

third()

def third():

raise RuntimeError("Failure")

try:

first()

except RuntimeError:

print("Handled")

A)NameError

B)TypeError

C)Handled

D)RuntimeError cannot propagate through functions

Q10. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.modules

B)sys.path

C)sys.imports

D)__module__

Q11. What happens?

def fun(a, b):

return a + b

print(fun(10, a=20))

A)SyntaxError

B)TypeError

C)30

D)20

Q12. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

A)TypeError

B)SyntaxError

C)20

D)30

Q13. What is the output?

def fun(a=10, b=20):

return a - b

print(fun(30))

A)20

B)30

C)-10

D)10

Q14. What happens here?

def fun(a):

return a

print(fun())

A)None

B)0

C)TypeError

D)NameError

Q15. What is the output?

def outer():

x = 10

def inner():

return x + 5



```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

10

B)10

10

C)Module loaded

Module loaded

10

D)ImportError

Q46. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B C

C)A C

D)B

Q47. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)NameError

B)TypeError

C)SyntaxError

D)It executes correctly

Q48. What happens here?

```
class MyError(Exception):
```

```
    pass
```

```
def test():
```

```
    raise MyError("Something went wrong")
```

```
try:
```

```
test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)Exception

B)Something went wrong

C)MyError

D)TypeError

Q49. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A)10

B)func(5)

C)5

D>Error

Q50. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)5

B)10

C)15

D>Error



variable = 10

fun()

A)NameError

B)UnboundLocalError

C)10

D)50

Q34. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Exception

Age is too low

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

15

D)InvalidAgeError

Age is too low

Q35. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)random cannot be imported

B)The local file can shadow the intended standard-library module

C)Python automatically renames the local file

D)Python will always reject it

Q36. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)TypeError

B)NameError

C)0

D)None

Q37. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)20

B)10

C)30

D)-10

Q38. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

A)SyntaxError

B)25

C)TypeError

D)15

Q39. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)A and B

B)Nothing

C)B

D)A

Q40. What will be printed?

```
def make_counter():  
    count = 0
```



```
try:
    if x < 0:
        raise ValueError("negative")
    return x
except ValueError:
    return 100
finally:
    print("cleanup")
```

print(fun(-5))

- A)100
- cleanup
- B)cleanup
- 100
- C)negative
- cleanup
- D)cleanup
- 5

Q20. What will happen?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal y
        y = 20
```

```
    inner()
```

outer()

- A)SyntaxError
- B)NameError
- C)10
- D)20

Q21. What will happen?

```
def fun(n):
    if n == 0:
        return 1
    return n * fun(n)
```

print(fun(3))

- A)RecursionError
- B)TypeError
- C)6
- D)3

Q22. Which object keeps track of modules that have already been

imported during the current Python process?

- A)sys.path
- B)sys.imports
- C)__module__
- D)sys.modules

Q23. What will be the output?

```
import math
```

print(math.sqrt(25))

- A)sqrt(25)
- B)25
- C)5.0
- D)5

Q24. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

- A)Only Hello
- B)x
- display
- C)10
- Hello
- D>Error

Q25. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

print(fun(3))

- A)None
- B)6
- C)3
- D)5

Q26. What happens here?



print(x)

A)20

10

B)20

30

C)30

30

D)30

20

Q6. What will be printed?

x = 10

f = lambda: x

x = 20

print(f())

A)10

B)NameError

C)20

D)None

Q7. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

A)TypeError

B)Handled

C)NameError

D)RuntimeError cannot propagate through functions

Q8. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

A)SyntaxError

B)TypeError

C)20

D)30

Q9. if test.py has
print("A")

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

A)Nothing

B)B

C)A

D)A followed by B

Q10. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

A)A

B)A B

C)Nothing

D)B

Q11. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

A)[10]

[10, 10]

B>Error

C)[10, 10]

[10]

D)[10]

[10]

Q12. What will normally happen?



D)A

Q45. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

- A)Python will always reject it
- B)The local file can shadow the intended standard-library module
- C)Python automatically renames the local file
- D)random cannot be imported

Q46. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)negative
cleanup
- B)cleanup
100
- C)cleanup
-5
- D)100
cleanup

Q47. What is the purpose of this statement?

if __name__ == "__main__":

- A)It prevents all imports
- B)It prevents syntax errors
- C)It identifies code intended to run when the file is executed directly
- D)It creates the main function

Q48. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:
```

```
print("A")  
except Exception:  
    print("B")
```

- A)Nothing
- B)B
- C)A B
- D)A

Q49. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)It executes correctly
- B)NameError
- C)SyntaxError
- D)TypeError

Q50. What happens here?

```
from math import sqrt
```

print(sqrt(36))

- A)6
- B)NameError
- C)36
- D)6.0



D)Error

Q32. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

A)UnboundLocalError

B)15

C)10

D)5

Q33. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)None

B)NameError

C)0

D)TypeError

Q34. What happens when this code runs?

```
def first():
```

```
    second()
```

```
def second():
```

```
    third()
```

```
def third():
```

```
    raise RuntimeError("Failure")
```

```
try:
```

```
    first()
```

```
except RuntimeError:
```

```
    print("Handled")
```

A)TypeError

B)Handled

C)NameError

D)RuntimeError cannot propagate through functions

Q35. What will be printed?

```
def make_counter():
```

```
    count = 0
```

```
    def counter():
```

```
        nonlocal count
```

```
        count += 1
```

```
        return count
```

```
    return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

3

4

B)1

1

1

1

C)1

2

1

3

D)0

1

0

2

Q36. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)Nothing

B)A B

C)A



numbers)

)

)

print(result)

A)[2, 4]

B)[2, 4, 6, 8, 10]

C)[1, 3, 5]

D)[4, 8]

Q19. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

A)20

B)30

C)SyntaxError

D)TypeError

Q20. What will happen?

def fun(a, b=10):

return a + b

print(fun(b=5, a=20))

A)TypeError

B)15

C)SyntaxError

D)25

Q21. What is the output?

def outer():

x = 10

def inner():

return x + 5

return inner

f = outer()

print(f())

A)None

B)NameError

C)10

D)15

Q22. Why is

try:

print(10 / 0)

except Exception:

print("A")

except ZeroDivisionError:

print("B")

codes's second handler never reached?

A)Exception is a superclass of ZeroDivisionError

B)except blocks execute randomly

C)ZeroDivisionError occurs before Exception

D)ZeroDivisionError is not an exception

Q23. What will be the output?

x = 10

def outer():

x = 20

def inner():

global x

x = 30

inner()

print(x)

outer()

print(x)

A)20

30

B)20

10

C)30

20

D)30

30

Q24. What will be the output?

x = 10

def fun():

print(x)

x = 20

fun()

A)UnboundLocalError

B)NameError

C)20



```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

```
outer()
```

- A)30
- B)NameError
- C)20
- D)10

Q7. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

- A)Error
- B)10
- C)5
- D)func(5)

Q8. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)A
- B)B
- C)A and B
- D)Nothing

Q9. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)10
- B)20
- C)30
- D)-10

Q10. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)Both import only sqrt
- B)First provides the module namespace; second imports sqrt directly
- C)There is no difference
- D)First imports only sqrt

Q11. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

- A)A, and execution ends normally
- B)A, followed by propagation of the same exception
- C)No Output
- D)B

Q12. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

```
print(fun())
```

- A)A
- B



Q45. What happens here?

try:

```
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

A)ValueError

B)Nothing

C)Wrong Value

D)e

Q46. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It creates the main function

B)It identifies code intended to run when the file is executed directly

C)It prevents syntax errors

D)It prevents all imports

Q47. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)10

B)24

C)9

D)0

Q48. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```

```
print(b)
```

A)[10, 10]

[10]

B)[10]

[10, 10]

C)Error

D)[10]

[10]

Q49. What will be printed?

try:

```
    x = int("10")
except ValueError:
    print("A")
```

else:

```
    print("B")
```

finally:

```
    print("C")
```

A)A

B)B C

C)A C

D)B

Q50. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)sqrt(25)

B)5.0

C)5

D)25



```
print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
```

A)SyntaxError

B)B

C)A

D)A B

Q32. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

A)NameError

B)TypeError

C)It executes correctly

D)SyntaxError

Q33. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)TypeError

C)SyntaxError

D)20

Q34. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

A)SyntaxError

B)25

C)TypeError

D)15

Q35. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)NameError

B)36

C)6.0

D)6

Q36. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)None

B)Hello

C)Hello None

D>Error

Q37. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

```
print(fun())
```

A)A

B

C

D

10

B)A

B

D

10

C)A

B

10

D

D)A

C

D

20



C)NameError

D)None

Q19. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)Only Hello

B)10

Hello

C)Error

D)x

display

Q20. What will be printed?

```
try:
```

```
    print(10 / 0)
```

```
except ZeroDivisionError:
```

```
    print("A")
```

```
except Exception:
```

```
    print("B")
```

A)A

B)Nothing

C)A B

D)B

Q21. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)None

B)6

C)5

D)3

Q22. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)NameError

B)10

C)None

D)20

Q23. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)15

B>Error

C)5

D)10

Q24. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():
```

```
    raise MyError("Something went  
wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)MyError

B)Something went wrong

C)Exception

D)TypeError

Q25. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)Python automatically renames the local file

B)random cannot be imported

C)The local file can shadow the intended standard-library module

D)Python will always reject it



reached?

- A) ZeroDivisionError occurs before Exception
- B) Exception is a superclass of ZeroDivisionError
- C) except blocks execute randomly
- D) ZeroDivisionError is not an exception

Q7. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

- A) [2, 4, 6, 8, 10]
- B) [2, 4]
- C) [4, 8]
- D) [1, 3, 5]

Q8. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A) TypeError
- B) SyntaxError
- C) 20
- D) 30

Q9. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the

same Python process, what is normally printed?

- A) Module loaded
- Module loaded
- 10
- B) 10
- 10
- C) ImportError
- D) Module loaded
- 10

Q10. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        global x
        x = 30
```

```
    inner()
    print(x)
```

```
outer()
print(x)
```

- A) 20
- 10
- B) 30
- 30
- C) 30
- 20
- D) 20
- 30

Q11. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A) 10
- B) 30
- C) -10
- D) 20

Q12. What is the output?

```
def fun(x):
    x += 10
```



Q45. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Hello
- B)Hello None
- C>Error
- D)None

Q46. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

- A)10
- B)20
- C)30
- D)NameError

Q47. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

```
A)B
```

```
B)A C
```

```
C)B C
```

```
D)A
```

Q48. What is the output?

```
def fun(a=10, b=20):
```

```
    return a - b
```

```
print(fun(30))
```

```
A)-10
```

```
B)30
```

```
C)20
```

```
D)10
```

Q49. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

```
A)10 followed by Finally
```

```
B)Finally only
```

```
C)10 only
```

```
D)Finally followed by 10
```

Q50. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

```
A)TypeError
```

```
B)NameError
```

```
C)SyntaxError
```

```
D)It executes correctly
```



Q31. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

A)A

B

D

10

B)A

B

C

D

10

C)A

C

D

20

D)A

B

10

D

Q32. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)NameError

B)UnboundLocalError

C)50

D)10

Q33. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents all imports

B)It identifies code intended to run when the file is executed directly

C)It prevents syntax errors

D)It creates the main function

Q34. What is the output?

```
def fun(x):  
    return x * 2
```

a = fun

print(a(5))

A>Error

B)5

C)10

D)func(5)

Q35. What will normally happen?

import math

import math

A)ImportError

B)The module is normally loaded and executed twice

C)The module is normally loaded once and reused

D)SyntaxError

Q36. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

print(result)

A)9

B)24

C)0

D)10

Q37. What is the output?



```
print(f())
```

- A)10
- B)None
- C)NameError
- D)20

Q20. Which object keeps track of modules that have already been imported during the current Python process?

- A)__module__
- B)sys.imports
- C)sys.path
- D)sys.modules

Q21. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

- A)Exception is a superclass of ZeroDivisionError
- B)ZeroDivisionError occurs before Exception
- C)except blocks execute randomly
- D)ZeroDivisionError is not an exception

Q22. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

- A)10
- B)5
- C)15
- D)UnboundLocalError

Q23. What will happen?

```
def fun(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return n * fun(n)
```

```
print(fun(3))
```

- A)6
- B)TypeError
- C)RecursionError
- D)3

Q24. Suppose sample.py contains:

```
x = 10
```

```
def display():
```

```
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
```

```
sample.display()
```

- A)10
- Hello
- B)x
- display
- C>Error
- D)Only Hello

Q25. What will be the output?

```
x = 10
```

```
def fun():
```

```
    print(x)
```

```
    x = 20
```

```
fun()
```

- A)UnboundLocalError
- B)20
- C)10
- D)NameError

Q26. What will be printed?

```
def make_counter():
```

```
    count = 0
```



```
print("cleanup")
```

```
print(fun(-5))
```

A)cleanup

-5

B)negative

cleanup

C)cleanup

100

D)100

cleanup

Q7. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError

B)10

10

C)Module loaded

10

D)Module loaded

Module loaded

10

Q8. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

A)There is no difference

B)Both import only sqrt

C)First imports only sqrt

D)First provides the module namespace; second imports sqrt directly

Q9. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)NameError

B)20

C)SyntaxError

D)10

Q10. What happens here?

```
def fun(a):
```

```
    return a
```

```
print(fun())
```

A)TypeError

B)0

C)None

D)NameError

Q11. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)Nothing

B)A

C)A B

D)B

Q12. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5

B)5.0

C)sqrt(25)

D)25



C)sys.imports
D)sys.modules

Q47. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

A)A
C
D
20
B)A
B
10
D
C)A
B
D
10
D)A
B
C
D
10

Q48. What is the output?

f = lambda x: x + 5

print(f(10))

A)10
B)15
C)5
D)Error

Q49. Suppose sample.py contains:

x = 10

```
def display():  
    print("Hello")
```

What will this print?

import sample

```
print(sample.x)  
sample.display()
```

A)Only Hello
B)10
Hello
C)x
display
D)Error

Q50. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

print(fun(30))

A)10
B)20
C)-10
D)30



C)Nothing, because custom exceptions cannot be caught

D)InvalidAgeError

Age is too low

Q34. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

A)RuntimeError

B)20

C)None

D)10

Q35. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

A)B

B)A B

C)Nothing

D)A

Q36. What will be printed?

from functools import reduce

numbers = [1, 2, 3, 4]

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

print(result)

A)10

B)24

C)9

D)0

Q37. What will be printed?

```
def fun(a, b):
```

return a + b

print(fun(10, b=20))

A)SyntaxError

B)TypeError

C)30

D)20

Q38. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

A)cleanup

100

B)100

cleanup

C)negative

cleanup

D)cleanup

-5

Q39. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

A)e

B)ValueError

C)Wrong Value

D)Nothing

Q40. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)UnboundLocalError



```

    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
A)SyntaxError
B)B
C)A B
D)A

```

Q22. What will be printed?

```

def make_counter():
    count = 0

    def counter():
        nonlocal count
        count += 1
        return count

    return counter

```

```

a = make_counter()
b = make_counter()

```

```

print(a())
print(a())
print(b())
print(a())

```

A)1

2

3

4

B)0

1

0

2

C)1

2

1

3

D)1

1

1

1

Q23. What is the output?

```

def outer():
    x = 10

```

```

def inner():
    return x + 5

```

```

    return inner

```

```

f = outer()

```

```

print(f())

```

A)15

B)None

C)NameError

D)10

Q24. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```

import random

```

A)random cannot be imported

B)The local file can shadow the intended standard-library module

C)Python automatically renames the local file

D)Python will always reject it

Q25. What happens when this function is called?

```

def fun(a=10, b):
    return a + b

```

A)SyntaxError

B)It executes correctly

C)NameError

D)TypeError

Q26. What happens here?

```

def fun(a):
    return a

```

```

print(fun())

```

A)NameError

B)TypeError

C)None

D)0

Q27. What will happen?

```

def fun(n):
    if n == 0:
        return 1

```



C)A

D)Nothing

Q8. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

a = fun()

b = fun()

print(a)

print(b)

A)Error

B)[10, 10]

[10]

C)[10]

[10]

D)[10]

[10, 10]

Q9. What is the output?

```
def fun(x):  
    return x * 2
```

a = fun

print(a(5))

A)Error

B)func(5)

C)10

D)5

Q10. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

print(outer())

A)15

B)UnboundLocalError

C)10

D)5

Q11. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

A)UnboundLocalError

B)NameError

C)10

D)20

Q12. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents syntax errors

B)It identifies code intended to run when the file is executed directly

C)It prevents all imports

D)It creates the main function

Q13. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)A, followed by propagation of the same exception

B)No Output

C)A, and execution ends normally

D)B

Q14. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

print(fun())

A)10 only

B)Finally followed by 10

C)10 followed by Finally

D)Finally only

